HealthVault

Secure Healthcare Data Sharing Platform

End-to-end encrypted • Patient-controlled consent • Hash-chain audit trail

Indian Institute of Information Technology, Dharwad

What Problem are we solving ?

“ This project is solving the problem of secure medical-record sharing between patients, doctors, and admins without losing privacy, consent control, or traceability. ”

What is HealthVault?

A production-grade, privacy-preserving web application for sharing medical records between patients, doctors, and administrators.

Patient

- Upload & manage records
- Grant / revoke doctor access
- Set expiry on consents
- View own audit logs

Doctor

- View consented records only
- MFA required at login
- Attribute-filtered access
- View own audit logs

Admin

- System statistics dashboard
- Manage all users & roles
- Verify blockchain integrity
- View all audit logs

The Four Core Security Questions

Every sensitive action is evaluated against four questions:

01

Who is this user?

Identity verified via bcrypt passwords + JWT tokens + TOTP MFA

02

Are they allowed?

Role-Based Access Control (Patient / Doctor / Admin) restricts every endpoint

03

Has the patient consented?

Patient explicitly grants per-doctor, per-record, time-limited consent

04

Can we prove access happened?

Tamper-evident SHA-256 audit chain records every sensitive action

Login Security – Proving Who You Are

Two Tokens Issued:

Access Token

Lifetime: 15 minutes

Used for all API calls. Short-lived so stolen tokens expire quickly.

Refresh Token

Lifetime: 7 days

Used only to get a new access token when the old one expires.

Multi-Factor Authentication (MFA)

Required for Doctors and Admins. A stolen password alone is not enough.

1 — 2 — 3 — 4 — 5

Enter Password
User submits email + password

Temp Token
Server returns mfaRequired: true + temp token

OTP Code
User reads 6-digit code from authenticator app

Verify TOTP
Server checks code against shared secret

Full JWT
Access + Refresh tokens issued on success

How TOTP Works

- Phone + server share a secret key at setup (QR code scan)
- Both independently compute a 6-digit code from the current time (30-second windows)
- If codes match → access granted. Attacker needs both password AND physical device

Record Encryption – AES-256-GCM Envelope

Encryption Steps

1. **Random DEK**

   32-byte Data Encryption Key generated per record

2. **Encrypt Record**

   AES-256-GCM encrypts data → ciphertext + IV + auth tag

3. **Encrypt DEK**

   Master key (from env) encrypts the DEK → stored in DB

4. **Discard DEK**

   Raw DEK wiped from memory – never stored in plaintext

Why AES-256-GCM?

Confidentiality

256-bit key makes brute force infeasible

Integrity

Auth tag detects any tampering during decryption

Envelope model

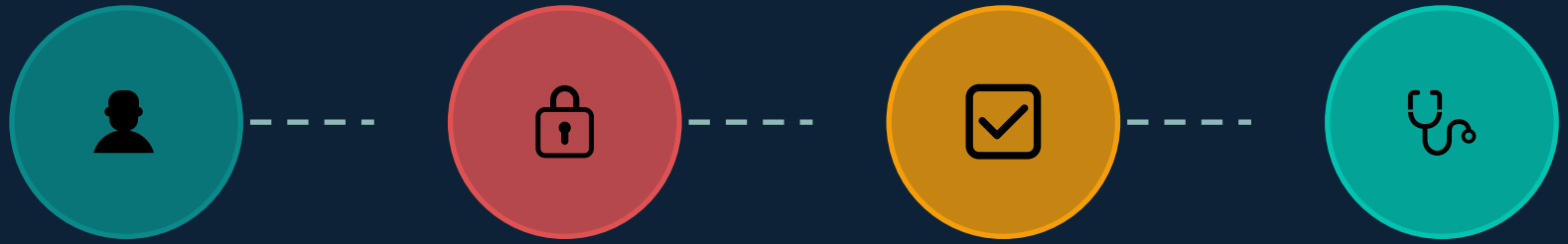
Each record has its own key — one leak ≠ all records exposed

Master key isolation

lives in env vars, not in the database

Patient Consent System

Even authenticated doctors CANNOT access records without explicit patient consent.



Patient Uploads Record

Record Locked by Default

Patient Grants Consent

Doctor Can Now View

Consent Rules

Scope:	Per-record or all records
Duration:	With or without expiry date
Revocable:	Patient can revoke at any time
Server-enforced:	Checked server-side on every request

Access Control: RBAC

Role-Based Access Control (RBAC)

Action	Patient	Doctor	Admin
Upload records	☑	✗	✗
View own records	☑	—	✗
View patient records	—	☑ w/consent	✗
Grant/revoke consent	☑	✗	✗
Verify Hash-chain	✗	✗	☑
Manage users	✗	✗	☑

Audit Logging & Tamper-Evident Chain

What Gets Logged

- Login / logout
- Record upload
- Record view / download
- Consent granted / revoked
- Record deletion

Each log captures: User • Record • Timestamp • IP • Action

SHA-256 Hash Chain

Tamper Detection

If Block #2 is modified → its hash changes → Block #3's previousHash no longer matches → verifyChain() fails → tamper detected

Note: This is an internal hash chain – not a decentralized blockchain. Provides tamper evidence, not public consensus.

API Hardening & File Upload Security

Security Middleware Pipeline

`helmet()`	Safer HTTP headers (CSP, HSTS)
`cors()`	Restricts allowed frontend origins
`rateLimit()`	100 req / 15 min per IP
`auth()`	JWT token verification
`rbac()`	Role authorization check
`consentCheck()`	Active consent verification

File Upload Security (Multer)

- Upload cap enforced
- Only safe file types accepted
- File encrypted before touching disk
- Raw upload deleted immediately after
- IV + auth tag + encrypted DEK in Postgres

End-to-End: Doctor Accessing a Record

1

Patient Upload

Backend encrypts record with AES-256-GCM before saving

2

Stored Encrypted

Ciphertext on disk, metadata in Postgres — unreadable by default

3

Consent Granted

Patient explicitly grants doctor access (with optional expiry)

4

Doctor Logs In

Password verified + MFA TOTP code checked → JWT issued

5

Request Record

JWT validated → Role checked → Consent verified → Attributes checked

6

Decrypt & Return

Only now does backend decrypt and return the record to the doctor

7

Audit Written

Action logged with user, record, IP, timestamp → added to hash chain

What Works Well & Important Caveats

Strengths

- ✓ Passwords hashed with bcrypt
- ✓ Doctors & admins require MFA
- ✓ Records encrypted before storage (AES-256-GCM)
- ✓ Patient consent enforced server-side
- ✓ Every sensitive action is audited
- ✓ Audit records chained for tamper detection
- ✓ Routes protected by auth + role middleware
- ✓ Common web protections enabled (helmet, cors, rate-limit)

Important Caveats

ABE is not cryptographic

It is application-level attribute rule checking, not real crypto ABE

JWT in localStorage

Common practice, but weaker vs XSS than HttpOnly cookies

Internal blockchain only

SHA-256 chain inside the app – not decentralized

Master key in env vars

Fine for dev; production should use KMS / secret manager

Redis not security-critical

Present in stack but not central to current auth logic

HealthVault – Security Summary

A layered security model that combines:

Identity

bcrypt + JWT

MFA

TOTP required for doctors & admins

Roles

RBAC restricts every endpoint

Consent

Patient controls doctor access

Attributes

Extra policy filter post-consent

Encryption

AES-256-GCM at rest

Audit

Every action logged

Chain

Tamper-evident hash chain

Security is not just about blocking bad access — it's about making all access traceable and accountable.

Thank you

Security is not just about blocking bad access — it's about making all access traceable and accountable.